

Day 10: Generative AI at the edge

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

- ▶ Days 1 to 9 used models with thousands or millions of parameters. A small language model has billions of parameters. Its weights fill a large part of the RAM of the Raspberry Pi 5.
- ▶ A language model writes one token after the other. The speed of a chat is the number of tokens in each second, and the memory bandwidth limits it more than the arithmetic.
- ▶ A local model keeps the data on the device and works with no network. It can also give a wrong answer with confidence, and it uses much power and heat.
- ▶ In the lab you run two models of different size with Ollama, measure them with the method of Day 9, call a model from Python with structured output, and build one small application.

After this day you can:

- ▶ Estimate the RAM that a small language model needs from its parameters, its precision, and its context length.
- ▶ Run small language models on the Raspberry Pi and measure tokens per second.
- ▶ Call a small language model from Python with structured output and function calling.
- ▶ Explain retrieval-augmented generation and vision-language models at the edge.
- ▶ State when a small language model is the wrong tool.

Part 1 Small language models from a systems view

Exercise: calculate the RAM of a 4-bit model with 3 billion parameters

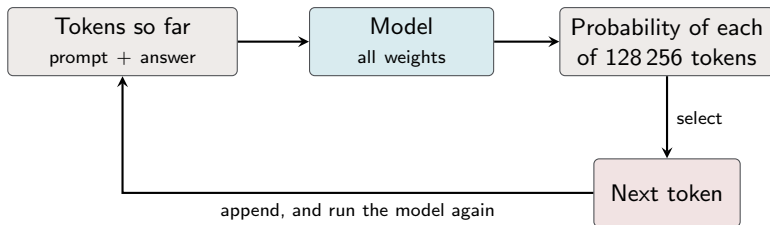
Part 2 Tools and runtimes

Exercise: select a model and a quantization level for three applications

Part 3 Applications and limits

Exercise: decide for four tasks: a classifier, a rule, or a language model

A language model writes one token at a time



- ▶ A classifier of Days 1 to 9 runs once for one input. A language model runs once for each token of its answer.
- ▶ An answer of 100 tokens needs 100 runs of the model. The speed is a rate: tokens in each second.
- ▶ The model stops at a stop token, or at a limit of the runtime.

Key point: The cost of an answer grows with its length. A short answer is a fast answer.

- ▶ A tokenizer cuts the text into tokens: words, parts of words, and signs. The model sees only the numbers of the tokens.
- ▶ The vocabulary of Llama 3.2 has 128 256 tokens. The tokenizer joins small parts with 280 147 merge rules (byte-pair encoding).
- ▶ The GGUF file of the model contains the tokenizer. You need no other file.
- ▶ The context is the number of tokens that the model can see: the prompt and the answer. Llama 3.2 is trained for 131 072 tokens.

Rate of a person

Reading: 5 to 7 words in each second

Rate of a model

About 8 to 10 tokens in each second

Key point: Count in tokens, not in words. The prompt, the context, the speed, and the price of a cloud model all use tokens.

Source: Vocabulary, merge rules, and context: the GGUF header of `llama3.2:3b`.
Reading rate: guide “SLMs at the Edge”, section 2.6.

Two phases: the prompt and the generation

Prompt evaluation (prefill)

- ▶ All tokens of the prompt go through the model together.
- ▶ The weights multiply a matrix of tokens: many operations for each byte that the processor reads.
- ▶ The result: the time to the first token.

Generation (decode)

- ▶ One new token for each run of the model.
- ▶ The weights multiply one vector: few operations for each byte.
- ▶ The result: the tokens in each second of the answer.

llama3.2:1b, Raspberry Pi 5	Tokens	Time	Rate
Prompt eval	32	1.64 s	19.46 tokens/s
Eval (the answer)	8	0.89 s	8.99 tokens/s

Ollama prints the two rates with `--verbose`. A benchmark of a language model gives the two rates and the two token counts.

Source: Lab “Small Language Models” of *Machine Learning Systems*.
The token count of the answer is 8.99×0.89 s, a calculation.

Value in the GGUF header	llama3.2:1b	llama3.2:3b
Layers (transformer blocks)	16	28
Width of the model, d	2048	3072
Attention heads	32	24
Key and value heads	8	8
Size of one head	64	128
Width of the MLP	8192	8192
Vocabulary	128 256	128 256
Parameters	1 235 814 432	3 212 749 888

- ▶ The names say 1B and 3B. The files have 1.24 and 3.21 billion parameters.
- ▶ Fewer key and value heads than attention heads: grouped-query attention. It makes the KV cache smaller (later in this part).

Source: The GGUF headers of the two files of the Ollama library, read for this course.

Count the parameters of the 3B model

Part	Calculation	Parameters
Attention: Q and output	$2 \times 3072 \times 3072$	18 874 368
Attention: K and V	$2 \times 3072 \times (8 \times 128)$	6 291 456
MLP: gate, up, down	$3 \times 3072 \times 8192$	75 497 472
Two norms	2×3072	6 144
One layer	sum of the four lines	100 669 440
28 layers	$28 \times 100 669 440$	2 818 744 320
Token table	$128 256 \times 3072$	394 002 432
Last norm and RoPE	$3072 + 64$	3 136
Total		3 212 749 888

- ▶ The MLP has 75 percent of the weights of a layer.
- ▶ The token table is also the output layer (tied weights). It has 12 percent of all parameters.

Source: Shapes: the GGUF header of `llama3.2:3b`.
The total is the sum of the tensors in the file.

Size in bytes = parameters × bits for each weight / 8

Format	Bits	1.24 B parameters	3.21 B parameters
float32	32	4.94 GB	12.85 GB
float16	16	2.47 GB	6.43 GB
Q8_0	8.5	1.31 GB	3.41 GB
Q4_0	4.5	0.70 GB	1.81 GB
Only the 4 bits	4	0.62 GB	1.61 GB
The file of the lab		1.32 GB, Q8_0	2.02 GB, Q4_K_M

- ▶ Training gives float16 or float32. A 3B model in float16 uses most of the 8 GB of the Raspberry Pi 5.
- ▶ A 4-bit format needs more than 4 bits: each block of weights also stores its scale.

Key point: Quantization makes a small language model fit on the board.

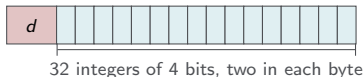
Source: Parameters: the GGUF headers.
File sizes: the Ollama registry.

Quantization in blocks: Q8_0 and Q4_0

Q8_0: one block of 32 weights = 2 + 32 bytes = 34 bytes



Q4_0: one block of 32 weights = 2 + 16 bytes = 18 bytes



- ▶ Each weight is $w = d \times q$. Each block of 32 weights has its own scale d .
- ▶ Q8_0: $34 \times 8/32 = 8.5$ bits for each weight. Q4_0: $18 \times 8/32 = 4.5$ bits.
- ▶ Day 6 used one scale for each tensor or for each channel. A small block gives a better fit to the weights, for some more bytes.

Source: Block definitions in `ggml-common.h` of `llama.cpp`.

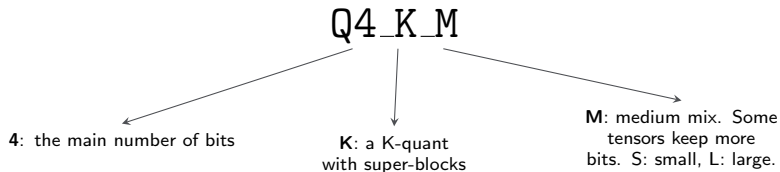
A K-quant groups 256 weights into a super-block of 8 blocks with 32 weights. Each block has a scale and a minimum of 6 bits. The super-block has two FP16 values for these scales: $w = a \times q + b$.

Type	Bytes of one super-block	Bytes	Bits for each weight
Q4_K	$2 + 2 + 12 + 128$	144	4.5
Q5_K	$2 + 2 + 12 + 32 + 128$	176	5.5
Q6_K	$2 + 16 + 64 + 128$	210	6.5625

- ▶ The bits per weight: $\text{bytes} \times 8 / 256$. Example: $144 \times 8 / 256 = 4.5$.
- ▶ The minimum b gives a better fit when the weights of a block are not centred on zero.
- ▶ The scales themselves are quantized: 6 bits, not 16. So the cost of 8 scales stays small.

Source: Block definitions in `ggml-common.h` of `llama.cpp`.

Read the name of a quantization level



Level	Advice of the guide
Q8_0	Almost no loss. For models of 1B or smaller.
Q5_K_M	A good compromise near 1B, for example for structured output.
Q4_K_M	The default of Ollama. For models of 2B and larger.
Q3, Q2	The quality drops fast. A 1B model at Q2 is often worse than a 360M model at Q8.

- ▶ A small model loses more quality from fewer bits than a large model.

Source: Guide “SLMs at the Edge”, section 2.2.
Published advice, not measurements of this course.

The tensors of llama3.2:3b, from its GGUF header:

Type	Tensors	Which tensors	Weights	Bytes
Q4_K	168	Q, K, output, gate, up; one half of V and down	2.422 B	1.362 GB
Q6_K	29	The token table; the other half of V and down	0.790 B	0.648 GB
F32	58	The norms and the RoPE values	0.0002 B	0.0007 GB
Sum	255		3.213 B	2.012 GB

- ▶ Mean: $2.012 \times 8 / 3.213 = 5.0$ bits for each weight, not 4.
- ▶ The tensors V and down of 14 of the 28 layers keep 6 bits. These tensors are the most sensitive to the error.
- ▶ The file has 2.019 GB. The other 7.8 MB are the header: most of it is the tokenizer.

Key point: Calculate the RAM from the size of the file, not from the name of the level.

Source: The GGUF header of llama3.2:3b.

Header: GGUF , version 3, counts
Metadata: 30 keys architecture, shapes, tokenizer, chat template
Tensor list: 255 names, shapes, types, offsets
Tensor data 2.012 GB

```
general.architecture      llama
general.name              Llama 3.2 3B Instruct
general.file_type         15 (Q4_K_M)
llama.block_count        28
llama.embedding_length    3072
llama.context_length      131072
llama.attention.head_count_kv 8
tokenizer.ggml.tokens     128256 strings
```

- ▶ One file has the weights, the tokenizer, and the chat template.
- ▶ llama.cpp, Ollama, and LM Studio read GGUF.
- ▶ The runtime can map the tensor data into memory: it loads only the pages that it reads.

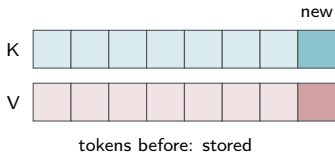
Source: Keys and counts: the GGUF header of `llama3.2:3b`, read for this course.



Example: `llama3.2:3b` with a context of 4096 tokens on a Raspberry Pi 5 with 8 GB. The width of “other” is not a measurement.

1. **Weights:** the size of the file.
 2. **KV cache:** it grows with the context length.
 3. **Other:** buffers of the runtime, the operating system, and your program.
- ▶ The source measured 3.24 GB in use with a 3.8B model loaded, and about 377 MB after Ollama stopped (no desktop).
 - ▶ Rule of the guide: usable model size $\approx (\text{total RAM} - \text{operating system and applications}) \times 0.7$. For 8 GB: $(8 - 0.4) \times 0.7 = 5.3$ GB for the weights and the cache.

Source: Memory values: lab “Small Language Models” of *Machine Learning Systems*.
Rule: guide “SLMs at the Edge”, section 2.4.



- ▶ Attention compares the new token with all tokens before it. It needs their keys K and values V in each layer.
- ▶ The runtime stores K and V. It does not calculate them again for each new token.
- ▶ The cache grows by one row for each token.

$$\text{Bytes} = 2 \times \text{layers} \times \text{KV heads} \times \text{head size} \times \text{context} \times \text{bytes for each value}$$

- ▶ The 2 counts K and V. Ollama stores the values in f16: 2 bytes. q8_0 uses about one half of this memory.
- ▶ The default context of Ollama is 4096 tokens. The runtime allocates the cache for this context.

Source: Ollama FAQ: the context window and the type of the K/V cache.

The KV cache of the two lab models

	llama3.2:1b	llama3.2:3b
Calculation for one token	$2 \cdot 16 \cdot 8 \cdot 64 \cdot 2$	$2 \cdot 28 \cdot 8 \cdot 128 \cdot 2$
Bytes for one token	32 768	114 688
4096 tokens (default)	0.13 GB	0.47 GB
32 768 tokens	1.07 GB	3.76 GB
131 072 tokens (maximum)	4.29 GB	15.03 GB
The weights (file)	1.32 GB	2.02 GB

- ▶ With the maximum context, the cache of the 3B model is 7 times its weights. It does not fit in 8 GB.
- ▶ With 24 KV heads instead of 8, the 3B cache would be 3 times as large: grouped-query attention saves this memory.

Key point: Set the context to the length that your application needs, not to the maximum of the model.

Source: Calculation of this course from the GGUF headers, with `f16` values.

For each new token, the model reads each weight once and does about 2 operations with it.

llama3.2:3b, one token	Value
Operations	$2 \times 3.21 \text{ B} = 6.4 \text{ GFLOP}$
Bytes read	2.02 GB (the file)
Intensity	$6.4/2.02 = 3.2 \text{ FLOP for each byte}$
At 5.3 tokens/s (published)	$6.4 \times 5.3 = 34 \text{ GFLOP/s}$

- ▶ Day 7: the peak of the 4 cores is 153.6 GFLOP/s. The generation uses less than one quarter of it.
- ▶ An intensity of 3.2 is low: the memory sets the limit, as for the matrix times vector of Day 7.
- ▶ The prompt is different: all its tokens use each weight. Its intensity is higher, so its rate is higher.

Source: Calculation of this course. Rate: lab “Small Language Models” of *Machine Learning Systems*.

The published rates agree with the memory limit

Model	Parameters	File	Eval rate	File $\times$ rate
llama3.2:1b	1.24 B	1.32 GB	8.99 tokens/s	11.9 GB/s
llama3.2:3b	3.21 B	2.02 GB	5.3 tokens/s	10.7 GB/s
Ratio 3B / 1B	2.6	1.53	1.70 (slower)	

- ▶ The 3B model has 2.6 times the parameters, but it is only 1.7 times slower. Its file is only 1.53 times as large.
- ▶ The product of the file size and the rate is almost the same for the two models: about 11 GB of weights in each second.
- ▶ So the bytes set the speed, not the parameters. Fewer bits give more tokens in each second.

Key point: Upper limit: tokens per second $\approx$ memory bandwidth / size of the file. Measure the rate in the lab to see your limit.

Source: Rates: lab “Small Language Models” of *Machine Learning Systems*, about 2 GB for the 3B file in the source. Products and ratios: calculations of this course.

Tokens per second: what is usable?

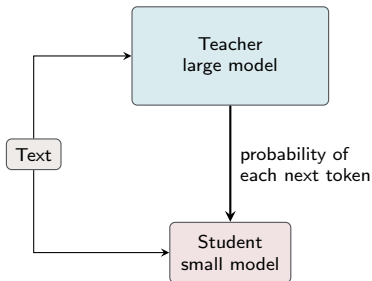
Use	Rate that you need	Why
Chat with a person	8 to 10 tokens/s	A person reads at this rate
A job with no person: summarize a log, classify a message	2 tokens/s can be enough	Nobody waits for each word

The total time of one answer (if the rates stay the same):

- ▶ A prompt of 500 tokens at 19.46 tokens/s: 25.7 s before the first token.
- ▶ An answer of 100 tokens: 11.1 s with the 1B model, 18.9 s with the 3B model.
- ▶ A reasoning model first writes a “think” text. The person waits for these tokens too.

Key point: The time to the first token depends on the prompt. A long prompt can cost more time than the answer.

Source: Uses and rates of a person: guide “SLMs at the Edge”, section 2.6. Times: calculations of this course with the published rates.



- ▶ The student learns the full probabilities of the teacher, not only the correct next token.
- ▶ These soft targets also show which wrong tokens are almost correct. This is more information for each example.
- ▶ Day 6 used the same idea for a classifier.

Llama 3.2 1B and 3B: the authors pruned the 8B model of Llama 3.1. Then the outputs of the 8B and 70B models were the targets for each token. The training used 9 trillion tokens.

Key point: A small model with good data and a good teacher can do a narrow task well. It does not have the breadth of the teacher.

Source: Hinton, Vinyals, and Dean, 2015. Llama 3.2: lab “Small Language Models” of *Machine Learning Systems*.

Good tasks for a model of about 1B

Summarize a text

Find the intent of a short command

Answer questions on a narrow topic

Extract fields from messy text

A basic conversation

Weak tasks

Reasoning in many steps

Facts that are not in the prompt

Exact arithmetic

Long and complex instructions

A guaranteed correct answer

- ▶ The guide: the question at the edge is not “how large a model fits”, but “how small a model still does the task”.
- ▶ Part 3 adds tools that help a small model: a schema for the output, function calls, and retrieval.

Key point: Test the model with your own prompts. A ranking on a public leaderboard does not tell you the result on your task.

Source: Guide “SLMs at the Edge”, sections 1 and 7.

Exercise: the RAM of a 4-bit 3B model

A model has 3.2 billion parameters, 28 layers, 8 key and value heads, and a head size of 128. The runtime stores the KV cache in f16 . The board is a Raspberry Pi 5 with 8 GB. The operating system uses 0.4 GB.

1. Calculate the size of the weights at 4 bits for each weight.
2. Calculate the bytes of the KV cache for one token.
3. Calculate the RAM of the weights and the KV cache for a context of 4096 tokens and for a context of 32 768 tokens.
4. Use the rule of the guide: $(8 - 0.4) \times 0.7 = 5.3$ GB. Which context fits?
5. The file of the lab has 2.02 GB, not your value of question 1. Why?
6. Give two changes that make the larger context fit.

Use $\text{GB} = 10^9$ bytes.

1. $3.2 \times 10^9 \times 4/8 = 1.6$ GB.
2. $2 \times 28 \times 8 \times 128 \times 2 = 114\,688$ bytes.
3. 4096 tokens: $114\,688 \times 4096 = 0.47$ GB, total $1.6 + 0.47 = 2.07$ GB.
32 768 tokens: $114\,688 \times 32\,768 = 3.76$ GB, total $1.6 + 3.76 = 5.36$ GB.
4. 4096 tokens fits with a large margin. 32 768 tokens is above 5.3 GB: it does not fit with the rule. The real file is larger too.
5. Each block stores its scales, and Q4_K_M keeps some tensors at 6 bits: 5.0 bits for each weight on average.
6. A q8_0 KV cache: 1.88 GB, total 3.48 GB. Or a shorter context, for example 16 384 tokens. Or a smaller model.

Key point: The weights are fixed. The context is your choice, and it can cost more RAM than the weights.

Part 1 Small language models from a systems view

Exercise: calculate the RAM of a 4-bit model with 3 billion parameters

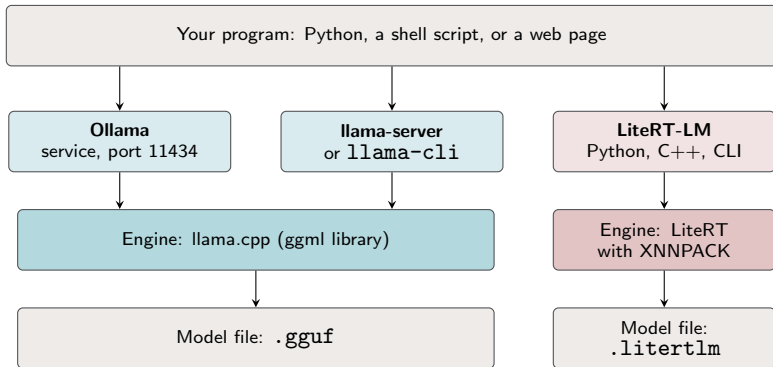
Part 2 Tools and runtimes

Exercise: select a model and a quantization level for three applications

Part 3 Applications and limits

Exercise: decide for four tasks: a classifier, a rule, or a language model

Three tools for a language model on a board



- ▶ Ollama and llama-server use the same engine and the same GGUF files. They differ in the packaging.
- ▶ LiteRT-LM is a second engine with its own file format. A GGUF file does not run in it.

Source: Guide “SLMs at the Edge”, section 3. Chapter “LiteRT-LM” of *Edge AI Engineering*.

- ▶ A C and C++ library with no dependencies. It has NEON kernels for Arm and kernels for x86. It reads GGUF files.
- ▶ You build it, or you download a release. On the Raspberry Pi 5 a build with 4 jobs needs about 10 minutes.
- ▶ No service and no model manager: you download the GGUF file and give its path.

Program	Use
llama-cli	A prompt or a chat in the terminal
llama-server	An HTTP server with an OpenAI-compatible API, port 8080
llama-bench	A benchmark of the prompt rate and the generation rate

```
llama-server -m llama3.2-3b-q4_k_m.gguf -c 4096 -t 4 --port 8080
```

-c: the context. -t: the threads. Give the context yourself: its default 0 means “the context of the model file”.

Source: llama.cpp documentation: [README](#) and [tools/server](#). Build time: [guide](#) “SLMs at the Edge”, section 5.2.

```
curl -fsSL https://ollama.com/install.sh | sh    # install the service
ollama pull llama3.2:1b                          # download a model
ollama run llama3.2:1b --verbose                  # chat, print the rates
ollama ps                                         # loaded models, RAM
```

- ▶ A service in Go around the engine of llama.cpp. It starts with the board and listens on port 11434.
- ▶ A model name has a tag: llama3.2:1b is the 1B model at Q8_0, llama3.2:3b the 3B model at Q4_K_M.
- ▶ The service keeps a model in RAM for 5 minutes after the last request (keep_alive). The next request does not load it again.
- ▶ A REST API for all programs, and a Python library that calls this API.

Key point: Ollama is the shortest way from an empty board to a model that answers. The lab uses it.

Source: Ollama documentation: README, API, and FAQ. Guide “SLMs at the Edge”, section 3.

- ▶ The runtime of Google for language models on LiteRT. LiteRT is the new name of TensorFlow Lite (Days 5 to 9).
- ▶ A `.litertlm` file holds the weights (int4 or int8), the tokenizer, and the settings.
- ▶ On the Raspberry Pi 5 it runs on the CPU with XNNPACK.
- ▶ The Python API has an `Engine`, which loads the model once, and a `Conversation`, which keeps the KV cache of the chat.

```
uv tool install litert-lm
litert-lm run --from-huggingface-repo=google/gemma-3n-E2B-it-litert-lm \
  gemma-3n-E2B-it-int4 --prompt="What is the capital of France?"
```

- ▶ Published on the Raspberry Pi 5: Gemma 4 E2B gives about 4 to 8 tokens/s on the CPU, similar to Ollama with a model of the same size.
- ▶ The same runtime runs on phones, in the browser, and on the board. Fewer models than Ollama: about 20.

Source: LiteRT-LM README. Chapter “LiteRT-LM” of *Edge AI Engineering*.

Compare the three tools

	Ollama	llama.cpp	LiteRT-LM
Setup	One command	A build or a release	<code>pip</code> or <code>uv</code>
Models	A library with many tags	Any GGUF file	About 20 <code>.litertlm</code> files
Interface	CLI, REST API, Python library	CLI, HTTP server, C library	Python, C++, Kotlin, Swift, CLI
Control	Options in each request	All settings	Engine and conversation
Measurement	<code>--verbose</code> , timing fields	<code>llama-bench</code>	Your own timer
Good for	A fast start, a service for programs	Control, a small memory, new models	One runtime from the board to a phone

- ▶ The guide: on the Raspberry Pi 5, Ollama first, and llama.cpp “for the last 10 percent”.
- ▶ A published test on the Raspberry Pi 5: llama.cpp was a little faster than Ollama (14.4 against 13.8 tokens/s, one 1.1B model).

Source: Guide “SLMs at the Edge”, sections 3, 5.2, and 8. Chapter “LiteRT-LM” of *Edge AI Engineering*.



	Server	Library
Examples	Ollama, llama-server	LiteRT-LM Engine, the C library of llama.cpp
Good	One model for many programs; change the runtime and keep the code	No second process; full control of the memory
Cost	One more process, the HTTP time	The model loads with your program

The Python library `ollama` is a client: the model runs in the service, not in your Python process.

One request to the Ollama API

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2:1b", "prompt": "Why is the sky blue?", "stream": false,
  "options": {"num_ctx": 2048, "num_thread": 4, "temperature": 0, "seed": 0}
}'
```

The answer is JSON. It has the text in response and these counters:

Field	Content
total_duration	The time of the complete request, in ns
load_duration	The time to load the model, in ns
prompt_eval_count	The tokens of the prompt
prompt_eval_duration	The time of the prompt, in ns
eval_count	The tokens of the answer
eval_duration	The time of the answer, in ns

Generation rate = $\text{eval_count} / \text{eval_duration} \times 10^9$. The prompt rate is the same calculation with the two prompt fields.

Source: Ollama API documentation.

The first request loads the model

Two requests with the same prompt (116 tokens) and 64 tokens of answer, llama3.2:1b, 4 threads:

Request	Total	Load	Prompt	Answer
First, after an unload	7.19 s	2.95 s	1.44 s	2.80 s
Second	4.61 s	0.58 s	1.21 s	2.81 s

- ▶ The load was 41 percent of the first request. The file was already in the page cache of the operating system.
- ▶ From a microSD card, a model of 1.5 GB loads in 10 to 20 s; from an NVMe SSD in 2 to 4 s (published).
- ▶ A benchmark: send one request first and do not count it, as the warm-up of Day 9.
- ▶ A product: keep the model loaded. `keep_alive: -1` keeps it, 0 unloads it at once.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only.
Load times of the board: chapter "LiteRT-LM" of *Edge AI Engineering*.

Set the options yourself

Setting	Ollama	llama.cpp	Documented default
Context	num_ctx	-c	Ollama: 4096 in the FAQ, 2048 in the Modelfile table. llama.cpp: the context of the model.
Threads	num_thread	-t, -tb	Set by the runtime
Answer length	num_predict	-n	Ollama: -1, no limit
Randomness	temperature, seed	--temp, --seed	Ollama: temperature 0.8
Stay in RAM	keep_alive	the server runs	Ollama: 5 minutes

- ▶ Two pages of one documentation give two defaults for the context. Do not depend on a default: give the value in the request.
- ▶ The context of Llama 3.2 is 131 072 tokens. With the default of llama.cpp, the KV cache of the 3B model needs 15 GB (Part 1).
- ▶ Temperature 0 and a fixed seed give the same answer for the same prompt: use them for a benchmark.

Source: Ollama documentation: FAQ, Modelfile, and API.
llama.cpp documentation of the CLI and the server.

Measured: the context costs RAM

llama3.2:3b, the RAM of the loaded model in `ollama ps`, for three values of `num_ctx`:

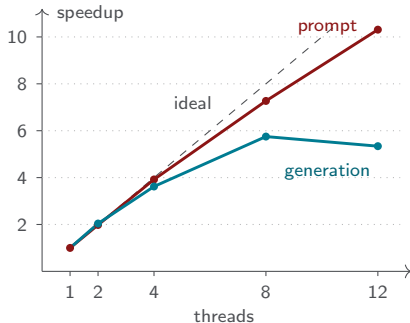
num_ctx	KV cache (Part 1)	File + KV cache	Measured	Difference
2048	0.235 GB	2.25 GB	2.32 GB	0.07 GB
8192	0.940 GB	2.96 GB	3.12 GB	0.16 GB
32 768	3.758 GB	5.78 GB	5.99 GB	0.21 GB

- ▶ The formula of Part 1 predicts the measured RAM. The difference is the buffers of the runtime: they also grow a little with the context.
- ▶ From 2048 to 32 768 tokens, the RAM grows by 119 262 bytes for each token. The formula gives 114 688: 4 percent less.
- ▶ On the 8 GB board, 32 768 tokens leave little RAM for the system and your program.

Key point: Calculate the RAM before the lab, then check it with `ollama ps` and `free`.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only.
The RAM of the model does not depend on the computer.

Measured: more threads



Threads	Prompt	Generation
1	25.5	6.2
2	50.4	12.5
4	99.7	22.3
8	185.0	35.4
12	262.5	32.9

Tokens/s, llama3.2:1b, median of 3 requests. The computer has 24 cores.

- ▶ The prompt is compute-bound: 12 threads give 10.3 times the rate.
- ▶ The generation is memory-bound: its gain stops at 8 threads, and 12 threads are slower.
- ▶ The Raspberry Pi 5 has 4 cores. The guide: threads = number of cores. Measure 1, 2, 3, and 4 threads in the lab.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only.

Measured: the prompt length

llama3.2:1b, 4 threads, 64 tokens of answer, median of 3 requests:

Prompt tokens	Prompt rate	First token	Generation rate
47	128.2 tokens/s	0.37 s	23.0 tokens/s
139	93.2 tokens/s	1.49 s	22.7 tokens/s
553	83.3 tokens/s	6.64 s	19.4 tokens/s

- ▶ The time to the first token grows faster than the prompt: a prompt 12 times as long needs 18 times the time.
- ▶ The generation is also slower with a long prompt. Each new token reads a larger KV cache.
- ▶ Retrieval (Part 3) puts long texts into the prompt. Measure with prompts of your real length.

Key point: A prompt rate without its prompt length is not a result. Write the two numbers.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only.

Measured: two model sizes on one CPU

4 threads, the same prompt of 116 tokens, 64 tokens of answer, median of 3 requests:

Model	File	Prompt rate	Generation rate	File $\times$ rate
llama3.2:1b	1.32 GB	99.8 tokens/s	22.94 tokens/s	30.3 GB/s
llama3.2:3b	2.02 GB	44.0 tokens/s	13.30 tokens/s	26.9 GB/s

- ▶ The 3B model generates 1.72 times slower on this computer. The published value of the Raspberry Pi 5 is 1.70 (Part 1).
- ▶ The prompt is 2.3 times slower: the prompt follows the operations, and the 3B model has 2.6 times the parameters.
- ▶ The rates of the work computer are about 2.5 times the rates of the board. The ratio between the two models is almost the same.

Key point: The generation follows the bytes of the file. The prompt follows the parameters.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only. Rates of the board: lab "Small Language Models" of *Machine Learning Systems*.

Tool	Model and level	Prompt	Generation	Prompt
Ollama	llama3.2:1b, Q8_0	19.46	8.99	32 tokens
Ollama	llama3.2:3b, about 2 GB	?	5.3	short
llama-bench	Qwen3.5 0.8B, Q8_0	about 134	about 10	?
llama-bench	Gemma 4 E2B, QAT Q4_K_M	about 37	about 7	?
LiteRT-LM	Gemma 4 E2B, int4	?	4 to 8	short

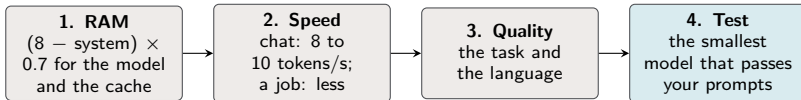
Rates in tokens/s. ?: not given in the source.

- ▶ Gemma 4 E2B through llama-server: 6.36 tokens/s; llama-bench: 6.63. A real request has the chat template and a filled KV cache.
- ▶ The rows use different models, prompts, and tools. Do not compare two rows as a benchmark.

Key point: A published rate is a starting point. Your measurement is the number for your decision.

Source: Lab “Small Language Models” of *Machine Learning Systems*. Chapters “From Ollama to llama.cpp” and “LiteRT-LM” of *Edge AI Engineering*.

Model selection for an 8 GB device



- ▶ The RAM is a hard limit. If the board swaps, the rate falls: use a smaller model or a smaller level.
- ▶ Take the instruct or chat version, not the base model.
- ▶ A reasoning model writes many tokens before the answer. On the board, disable the thinking mode or take a model with no thinking.
- ▶ Levels: Q8_0 up to 1B, Q5_K_M near 1B, Q4_K_M from 2B. A QAT or dynamic file is better at the same size.
- ▶ Read the licence of the model before a product.

Key point: The question at the edge: how small can the model be and still do the task?

Source: Guide “SLMs at the Edge”, sections 2, 7, and 8.

Model	Level	RAM	Rate	Note of the guide
SmolLM2 360M	Q8_0	0.5 GB	30 or more	Fast, little depth
Qwen3.5 0.8B	Q8_0	1 GB	15 to 25	Fast and coherent
Llama 3.2 1B, Gemma 3 1B	Q4_K_M	1 GB	8 to 20	The chat class
Qwen3.5 2B	dynamic Q4	1.6 GB	5 to 10	Better, still usable
Gemma 4 E2B	QAT Q4	3 GB	4 to 8	Images too
Llama 3.2 3B	Q4_K_M	2.2 GB	2 to 5	Jobs, slow for a chat
8B to 9B models	Q4_K_M	5 GB	1 to 2	Fits, but very slow

Rate: generation in tokens/s, CPU only, short context.

- ▶ The guide compiled these values from published benchmarks. Your values depend on the cooling, the context, and the prompt.
- ▶ The lab measures the two Llama 3.2 models of this table with their Ollama tags.
- ▶ Model names change fast. Check the list again before a project.

Source: Guide "SLMs at the Edge", section 5.1 (values compiled from published benchmarks).

Exercise: select a model for three applications

Use the candidates of the guide and the budget of 5.3 GB on a Raspberry Pi 5 with 8 GB. For each application, select a model and a quantization level. Give the rate that you need and the RAM.

1. **Home assistant.** A person speaks a short question and reads the answer on a screen. A speech model and a voice model run on the same board.
2. **Night report.** Each night the board summarizes the log of 20 sensors: about 6000 tokens of text. Nobody waits for it.
3. **Robot commands.** A command such as “move to the red box and stop” becomes a JSON object with 3 fields. The robot must start to move in less than 5 seconds.

For application 2, calculate the KV cache of Llama 3.2 3B for a context of 8192 tokens (Part 1: 114 688 bytes for each token).

Answer: select a model for three applications

1. Qwen3.5 0.8B at Q8_0, or Llama 3.2 1B. A chat needs 8 to 10 tokens/s, and these models give 8 to 25. About 1 GB leaves RAM for the speech and voice models.
2. Llama 3.2 3B at Q4_K_M: 2 to 5 tokens/s is enough for a job at night, and the summary is better. KV cache: $114\,688 \times 8192 = 0.94$ GB. Total $2.2 + 0.94 = 3.14$ GB, below 5.3 GB. A prompt of 6000 tokens takes minutes: this is acceptable at night.
3. A model of about 1B at Q5_K_M or Q8_0, with temperature 0, a JSON schema (Part 3), and a short num_predict. The prompt is short and the answer has about 30 tokens: at about 10 tokens/s, 3 s.

Key point: The application sets the rate and the context. Then select the smallest model that passes your test prompts.

Part 1 Small language models from a systems view

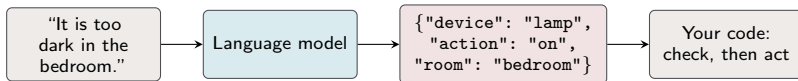
Exercise: calculate the RAM of a 4-bit model with 3 billion parameters

Part 2 Tools and runtimes

Exercise: select a model and a quantization level for three applications

Part 3 Applications and limits

Exercise: decide for four tasks: a classifier, a rule, or a language model



- ▶ A program cannot use a free text answer. It needs fields with known names and known values.
- ▶ A language model is good at the first step: many ways to say the same command become one meaning.
- ▶ Three methods connect the model to a program: a schema for the output, function calls, and retrieval of facts.

Key point: Use the model to understand the text. Use your code to decide and to act.

Source: Chapters “SLM: Basic Optimization Techniques” and “Experimenting with SLMs for IoT Control” of *Edge AI Engineering*.

```
schema = {"type": "object", "required": ["device", "action", "room"],
  "properties": {
    "device": {"type": "string",
      "enum": ["lamp", "fan", "heater", "door", "pump"]},
    "action": {"type": "string", "enum": ["on", "off", "open", "close"]},
    "room": {"type": "string",
      "enum": ["kitchen", "bedroom", "garage", "office"]}}}
r = ollama.generate(model="llama3.2:1b", prompt=text, format=schema,
  options={"temperature": 0})
```

- ▶ The runtime changes the schema into a grammar. At each step it allows only the tokens that keep the output valid.
- ▶ The output is then valid JSON with the correct keys and allowed values. The model still selects the values.
- ▶ llama.cpp: the schema does not go into the prompt. Describe the fields in the prompt too.
- ▶ `format="json"` is weaker: valid JSON, but any keys.

Source: Ollama API documentation: structured outputs. llama.cpp documentation: GBNF grammars.

Measured: valid JSON and correct JSON

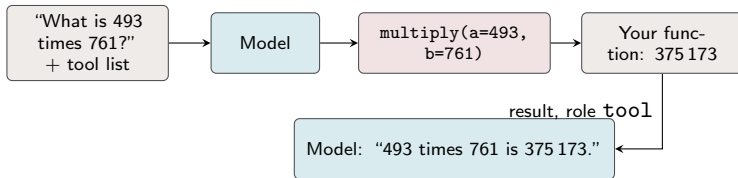
20 commands for a home controller. The prompt names the three fields and their values. Number of answers out of 20:

Mode	Model	Parses as JSON	Valid fields	Correct
Prompt only	1B	0	0	0
	3B	20	20	20
format="json"	1B	20	20	18
	3B	20	20	20
format=schema	1B	20	20	18
	3B	20	20	20

- ▶ The 1B model put each answer of the first mode into a Markdown code block. A strict parser rejects all 20. Without the block: 20 valid and 17 correct.
- ▶ With a format, the 1B model gives 2 wrong values: “It is too dark in the bedroom” becomes `lamp, off`. “The office is cold” becomes `heater, off`.
- ▶ The 3B model is correct 60 times out of 60, and 29 percent slower.

Key point: A schema makes the output valid. It does not make it correct.

Source: Experiment of this course: Ollama 0.32.6, CPU only, temperature 0.



```
tool = {"type": "function", "function": {"name": "multiply",  
    "parameters": {"type": "object", "required": ["a", "b"],  
        "properties": {"a": {"type": "integer"}, "b": {"type": "integer"}}}}}  
r = ollama.chat(model="llama3.2:1b", messages=msgs, tools=[tool])  
calls = r.message.tool_calls    # name and arguments, or empty
```

The model selects the function and the arguments. Your code runs the function. A second call of the model can write the answer as text.

Source: Ollama API documentation: tool calling. Chapter "SLM: Basic Optimization Techniques" of *Edge AI Engineering*.

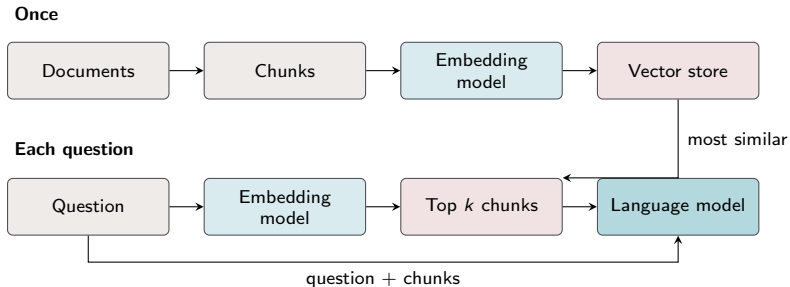
Temperature 0. Each task has 20 random problems (seed 7). Number of correct answers out of 20:

Task	1B	3B
Sum of two 2-digit numbers, direct answer	12	20
Product of two 3-digit numbers, direct answer	0	0
Product with the tool multiply: correct call	20	20

- ▶ The wrong products look correct: 151 120 (3B) for $220 \times 684 = 150\,480$.
- ▶ The 1B model got 8 of the 20 sums wrong, for example $63 + 18 = 843$.
- ▶ With the tool, each call had the correct function and the two numbers. In 33 of the 40 calls the numbers came as text ("653"): convert and check them in your code.

Key point: Let the model select the function. Let your code calculate.

Source: Experiment of this course: Ollama 0.32.6, x86 work computer, CPU only.



- ▶ The facts come from your documents at the time of the question. The model does not need new training.
- ▶ An embedding model maps a text to a vector. Similar texts give near vectors: compare them with the cosine similarity.
- ▶ Example: `nomic-embed-text`, 137M parameters, vectors of 768 values. It runs in Ollama on the same board.

Source: Lewis et al., 2020. Chapter "SLM: Basic Optimization Techniques" of *Edge AI Engineering*.

12 facts of a fictional greenhouse controller, 10 questions with a number or a name as the answer. Correct answers out of 10:

Prompt	1B	3B	Prompt tokens
The question only	0	0	47
The question and the 2 most similar facts	10	10	94
The question and all 12 facts	9	10	309

- ▶ The 2 most similar facts contained the answer for all 10 questions.
- ▶ With no facts, the 1B model said 7 times that it does not know. It invented 3 answers.
- ▶ With all 12 facts, the 1B model mixed two facts once: the fan starts at 70 percent, and the model said 60.

Source: Experiment of this course: Ollama 0.32.6, `nomic-embed-text`, x86 work computer, CPU only.

Prompt tokens: mean of the 10 questions.

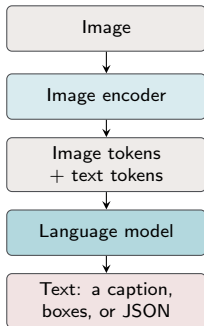
Each retrieved chunk makes the prompt longer. The prompt rate of the board sets the time before the first token.

Prompt	Tokens	At 19.46 tokens/s
The question only	47	2.4 s
The question and 2 facts	94	4.8 s
The question and 12 facts	309	15.9 s

- ▶ 19.46 tokens/s is the published prompt rate of llama3.2:1b on the Raspberry Pi 5 with a short prompt. The times are estimates: measure them in the lab.
- ▶ Keep the chunks short and k small. Retrieve only what the answer needs.
- ▶ The embedding of the question also takes time, but much less: the model is small and the question is short.

Key point: Retrieval trades RAM and prompt time for correct facts.

Source: Prompt tokens: the experiment of this course. Rate: lab “Small Language Models” of *Machine Learning Systems*.



- ▶ The image becomes many tokens. They are a long prompt for the language model.
- ▶ One model does many tasks: a caption, detection, text in the image (OCR). A prompt selects the task.
- ▶ Florence-2: 232M and 771M parameters. LLaVA-Phi-3: 3.8B parameters.

Published on the Raspberry Pi 5	Time
Florence-2 base: caption	16 to 17 s
Florence-2 base: object detection	20 to 41 s
Florence-2 base: segmentation	83 to 207 s
LLaVA-Phi-3: describe one image	almost 4 min

Source: Chapter "Vision-Language Models at the Edge" of *Edge AI Engineering*. Lab "Small Language Models" of *Machine Learning Systems*.



- ▶ The chapter connects a temperature sensor, a pressure sensor, a button, and three LEDs to the Raspberry Pi 5.
- ▶ The prompt has the readings and the rules in order of priority: the button (an alarm) first, then the temperature, then normal operation.
- ▶ The model answers with a JSON object that names one LED. If the program cannot parse the answer, the LEDs keep their state.
- ▶ A person can also give a command in a natural language, for example “turn on the yellow LED if the temperature is above 20 degrees”.

Key point: The rules of the prompt are a request, not a guarantee. Put the safety rules into your code.

Source: Chapter “Experimenting with SLMs for IoT Control” of *Edge AI Engineering*.

Guard	Example
A list of allowed actions	Only the functions and values of the schema exist
A check of each argument	The heater accepts 5 to 30 degrees, not 300
A rule for safety	The alarm button turns the red LED on, with no model in the path
A person confirms	Opening a door needs a “yes” from the user
A time limit	No answer in 10 s: keep the state, write a log line
A log	Each prompt, answer, and action, with the time

- ▶ Never run the text of a model as code or as a shell command.
- ▶ A document or a web page can contain an instruction, and the model can follow it. Your code checks each output.

Key point: The model proposes. The code decides.

- ▶ A language model writes a probable text. A probable text can be false, and it sounds as sure as a true one.
- ▶ Temperature 0 gives the same answer each time. It does not give the correct answer.

Question, no facts	Answer of llama3.2:1b	The fact
Below which light level does the light panel switch on?	"... usually around 10-20 lux."	3500 lux
At which humidity does the fan start?	"... when the relative humidity falls below 40%."	70 percent
How long must I hold the RESET button?	"You don't need to hold any buttons ..."	8 seconds

Countermeasures: retrieval, tools, a schema, and a test set.

Key point: Measure the accuracy of your application, as for a classifier: a test set, a count, and a target.

Source: Experiment of this course: Ollama 0.32.6, temperature 0. Chapter "SLM: Basic Optimization Techniques" of *Edge AI Engineering*.

The latency of one request

One question with a tool call, llama3.2:1b. The token counts are measured. The times are estimates for the Raspberry Pi 5 with the published rates: 19.46 tokens/s for the prompt, 8.99 for the answer.

Step	Tokens	Time
Call 1: the prompt with the tool list	165	8.5 s
Call 1: the tool call	26	2.9 s
Your function		0.0 s
Call 2: the new part of the prompt	96	4.9 s
Call 2: the answer as text	16	1.8 s
Total		18.1 s

- ▶ The tool list makes the first prompt long: a question of 5 words became 165 tokens.
- ▶ Call 2 is not always necessary: your program can show the result of the function. Then the total is 11.4 s.

Source: Token counts: experiment of this course. Rates: lab “Small Language Models” of *Machine Learning Systems*.

- ▶ During the generation, all 4 cores of the Raspberry Pi 5 run at almost 100 percent. This is the heavy load of Day 9: 7 to 10 W, 70 to 80 °C with the active cooler (published ranges).
- ▶ With Florence-2, the temperature went above 80 °C. A board that throttles loses 20 to 30 percent of its rate.

Estimate: one answer of 100 tokens, llama3.2:1b	Value
Time at 8.99 tokens/s	11.1 s
Energy at 7 to 10 W	78 to 111 J
Answers from a battery of 3.7 Wh (13 320 J)	120 to 171
The kit chip of Day 9 at 217 mW for 1 hour	781 J

One hour of the active kit chip uses the energy of 7 to 10 answers.

Key point: A language model is the most expensive way to make a decision on the board. Use it where nothing smaller works.

Source: Day 9, Part 3. Lab “Small Language Models” of *Machine Learning Systems*. Guide “SLMs at the Edge”, section 5. Energy: estimates of this course.

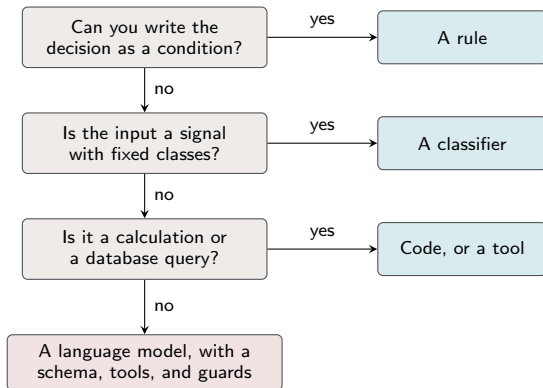
When a language model is the wrong tool

The task	The tool	Example
Fixed classes from a sensor or an image	A classifier (Days 1 to 9)	A keyword, a gesture, a cup or a bottle
An exact condition	A rule in code	Temperature above 31 °C: open the window
A calculation	Code, or a tool	A sum, a mean, a distance
Free text, many ways to say one thing	A language model	A spoken command, a summary of a log

- ▶ A classifier and a rule are fast, cheap, and they give the same answer each time. You can test them completely.
- ▶ A language model is flexible, but slow, large, and sometimes wrong.
- ▶ A good design often combines them: the model reads the text, a rule checks the result, and a classifier watches the sensor.

Key point: Fixed function first. A language model only for the part that needs language.

Source: Chapter “Classification of AI Applications”
of *Edge AI Engineering*.



Then check the budget of Part 2: RAM, the rate that the user needs, and the energy.

Exercise: a classifier, a rule, or a language model?

For each task, select a classifier, a rule, or a language model. Give one reason, and one number that supports it.

1. A door sensor sends “open” or “closed”. Send a message when the door is open for more than 5 minutes at night.
2. A camera at the door of a shop counts the people who come in.
3. A farmer sends a voice note: “the pump in the east field makes a strange noise since this morning”. Make a maintenance ticket with the device, the place, and the problem.
4. A battery sensor node must detect the word “stop” for one year on a small battery.

For task 3, write the JSON schema of the ticket with three fields.

Answer: a classifier, a rule, or a language model?

1. A rule: `if door == "open" and minutes > 5 and night`. It needs no model, it is exact, and you can test it completely.
2. A classifier, as a detector of people (Day 8). It runs at many frames in each second. A language model needs seconds for each image.
3. A language model: the text is free, and the same problem has many forms. Schema: `device` (enum of the devices), `place` (string), `problem` (string). One ticket in 10 to 20 s on the board is acceptable.
4. A classifier: keyword spotting on the microcontroller (Day 5). The chip of the kit sleeps at 0.79 mW (Day 9). A language model needs 7 to 10 W on the Raspberry Pi.

Key point: The task decides the tool. A language model is one tool among four.

- ▶ The weights of a model need parameters times bits per weight. The KV cache grows with the context length. The operating system and the other programs need the rest of the RAM.
- ▶ The prompt rate and the generation rate are two numbers. On a CPU, the memory bandwidth limits the generation rate.
- ▶ A runtime such as Ollama or llama.cpp runs a quantized model file in the GGUF format. The size of the model and its quantization level set the speed and the quality.
- ▶ Structured output, function calls, and retrieval make a language model useful in a program. A classifier or a rule is the better tool when the task has fixed classes.

Key point: Select the smallest model that does the task. Measure its tokens per second, its RAM, and its temperature on the device, and check its answers.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Raspberry Pi labs: “Small Language Models” and “Vision-Language Models”.
- ▶ *Edge AI Engineering*: “SLMs at the Edge”, “SLM: Basic Optimization Techniques”, “Vision-Language Models at the Edge”.

Papers

- ▶ “LLaMA: Open and Efficient Foundation Language Models”, Touvron et al., 2023.
- ▶ “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, Lewis et al., 2020.
- ▶ “Distilling the Knowledge in a Neural Network”, Hinton, Vinyals, and Dean, 2015.

Tools

- ▶ Ollama: <https://ollama.com>
- ▶ llama.cpp: <https://github.com/ggml-org/llama.cpp>
- ▶ LiteRT-LM: <https://github.com/google-ai-edge/LiteRT-LM>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.